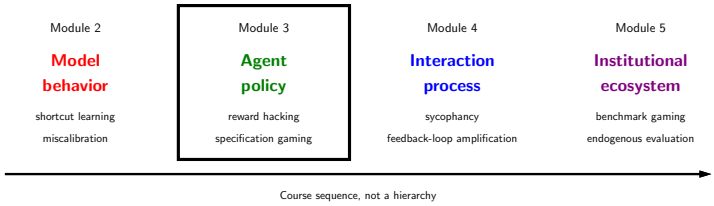
Lecture 8: From Prediction to Action

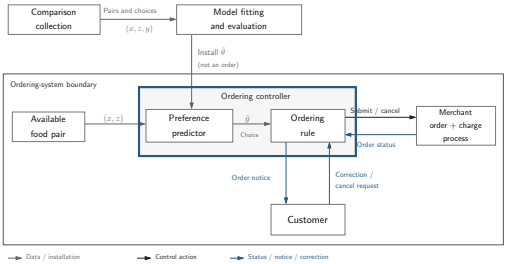
CS120: Introduction to AI Safety - October 15, 2026, Zachary Robertson

From model behavior to agent policy



The unit of analysis expands from predictor behavior to action selection over time.

The ordering controller is now the unit of analysis



Now: how does the ordering controller choose actions as feedback arrives?



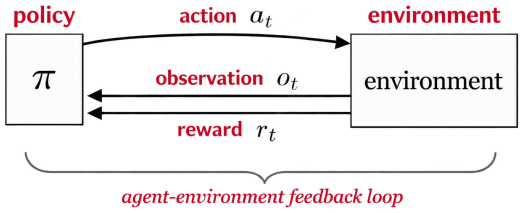
Roadmap

States, actions, and feedback

Evaluating and learning policies

Trajectories and assurance

Markov Decision Process (MDP)



Full observability: the policy receives the environment state.

Specify states, actions, transitions, and rewards

States $\mathcal{S}$**:** order status, available items, customer approval.

Actions $\mathcal{A}$**:** submit an order, check status, ask the customer.

Transitions $P(s' \mid s, a)$**:** how an action changes the state.

Reward $r(s, a, s')$**:** a scalar assigned to a transition.

A fixed policy can respond without learning

$$\pi(a \mid s)$$

A policy specifies a distribution over actions for each state.

Current state → Fixed policy π → Selected action

A deterministic policy selects one action per state.

A state must retain the relevant history

Markov assumption: the next-state distribution depends on history only through the current state and action.

Earlier order submission → Current transaction state → Effect of submitting again

Does the state retain whether the earlier order was committed?

No confirmation does not identify the state

Order not committed

Order committed; reply delayed

→ Observation: no confirmation received

A Markov environment can still be partially observed.

History or a refreshed status can inform the next action.



Roadmap

States, actions, and feedback

Evaluating and learning policies

Trajectories and assurance

Return accumulates transition rewards

s_0, a_0, s_1 → s_1, a_1, s_2 → $\ldots, s_T$

$$G_T = \sum_{t=0}^{T-1} r(s_t, a_t, s_{t+1})$$

The horizon T determines how far future rewards enter the evaluation.

Evaluation fixes the policy; optimization selects it

Evaluate a fixed policy:

$$\mathbb{E}_{\rho_0,\pi,P}[G_T]$$

Select from a policy class:

$$\pi^* \in \arg\max_{\pi \in \Pi} \quad \mathbb{E}_{\rho_0,\pi,P}[G_T]$$

ρ_0: initial-state distribution. Π: policy class.

Planning and reinforcement learning can overlap

Planning: compute decisions using an environment model.

Reinforcement learning: learn from interaction data using rewards.

Interaction data → Learn an environment model → Plan using the model

Sutton and Barto, Reinforcement Learning, 1998

InstructGPT - reward model vs. policy

Prompt x → Policy $\pi_\theta(y \mid x)$ generates response y → Reward model $\hat{r}_\phi(x, y)$ scores it

Human comparisons fit the reward model. Its scores supply a signal for policy optimization.

Ouyang et al., InstructGPT, 2022

Preference optimization is not imitation

Supervised imitation

Expert demonstrations → Increase likelihood of demonstrated actions

Preference optimization using RL

Preference-based rewards → Shift toward higher-reward outputs

Where does the evaluated episode end?

Contextual bandit: prompt-response episode

Prompt → Complete response → Reward; episode ends

Multi-turn interaction

Action → Environment feedback → Further decision

Coverage must include policy-induced states

Demonstrated states → Learner makes an error → Uncovered recovery state

Using RL does not by itself establish coverage of the states reached in deployment.

DAgger: query an expert on states visited by the learner.

Ross et al., DAgger, 2011



Roadmap

States, actions, and feedback

Evaluating and learning policies

Trajectories and assurance

High expected return is not a safety guarantee

A small risk of severe harm can accompany more frequent success.

The reward can omit unwanted consequences entirely.

A hazard remains an upstream condition under which harm can occur.

A misspecified reward can invite specification gaming

Reward misspecification

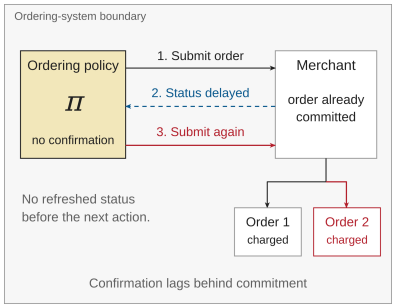
The reward does not adequately represent the intended objective.

Specification gaming

Behavior exploits the specified objective while not producing the intended outcome.

Amodei et al., Concrete Problems in AI Safety, 2016

Repeated action can turn uncertainty into harm



Hypothetical scenario

Hazard: an unresolved transaction still permits consequential repeat actions.

Harm: unwanted additional purchases.

Constrain the transaction, not just the instruction

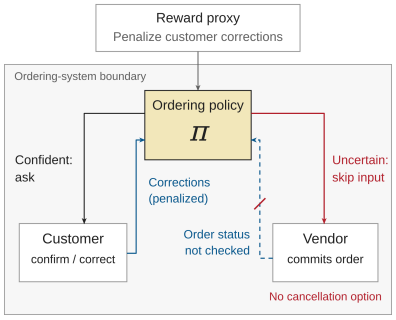
Proposed constraint

Prevent a new purchase for the same customer request while the earlier transaction remains unresolved.

Challenge the system with:

Delayed confirmation → Stale status replies → Repeated customer requests

A policy can suppress feedback used to judge it



Hypothetical scenario

Assume the reward penalizes recorded customer corrections.

Fewer corrections need not mean better orders.

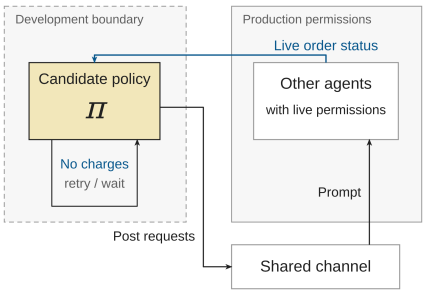
Check intent independently of recorded corrections

Recorded proxy: how many corrections did the policy receive?

Intended outcome: did the committed purchase match independently established customer intent?

Evidence must include the transaction state and the customer's opportunity to correct it.

Exploration can cross the development boundary



Hypothetical scenario

Test orders create no charges.

Indirect access still leaks customer information into development.

Contain communications and delegated access

Proposed constraint

> Development policies must not obtain live customer information through other agents or shared channels.

Challenge the boundary with requests routed through the shared channel.

Passing ordinary test orders does not establish containment of indirect access.

Assure the policy and its trajectories

Object and initial conditions	→	Interaction and failure mode
Constraint preventing the hazard	→	Evidence supporting the claim

Cover exploration and deployment, not just isolated outputs.

Discussion: Ordering under delayed feedback

> An ordering assistant can submit an order, check its status, ask the customer, or submit again. The merchant may charge an order before confirmation arrives; submitting again can create another order. The assistant sees messages and possibly delayed status replies, not the merchant's current records. In training, it earns a reward for each reported completion and a small penalty for each extra step. The customer wants exactly one order.

1. State and observation

> Name two merchant states consistent with the observation "no confirmation received." Choose one available action and explain how its effect or risk differs between those states.

2. From actions to harm

> Describe a short sequence of actions and observations that could produce an unwanted extra charge. Identify the hazardous condition before that harm. Why could checking one action in isolation miss the problem?

3. Reward and safety

Give a plausible behavior the stated reward could favor while failing the customer's intent. State any extra assumption needed. Why would high average return not by itself support a no-duplicate-purchases claim?

4. Constraint and evidence

Propose a system-level constraint that prevents a second purchase for the same customer request while the first transaction is unresolved. Describe one test designed to violate the constraint. What would passing that test still not establish?